

CENTRO REGIONAL UNIVERSITARIO CÓRDOBA IUA

HITO 5

Entrega Final — Documento Consolidado (Plan de Pruebas, Trazabilidad, Métricas y Cierre)

Proyecto: Power Strike

Materia: Ingeniería de Software II

Fecha de entrega: 16/06/2026

Equipo de trabajo

Mateo Brizuela — QA / Desarrollo

Ernesto Ponce — QC

Martín Mousist — QC

Fabrizio Posada — Documentador

Repositorio: github.com/MartinMousist/power-strike

Índice

1. Introducción
2. Plan de Pruebas
3. Diseño de Casos de Prueba
4. Ejecución y Resultados de los Tests Automatizados
5. Aplicación de TDD (Test-Driven Development)
6. Reporte de Defectos
7. Evolución del Repositorio durante el Hito 4
8. Análisis de Riesgos — FMEA
9. Pruebas Basadas en Estados
10. Conclusiones

1. Introducción

1.1 Contexto del Proyecto

Power Strike es un sistema web para la gestión integral de un gimnasio, desarrollado como Trabajo de Campo de la materia Ingeniería de Software II. El sistema permite administrar usuarios (clientes, entrenadores y administradores), actividades del gimnasio, asistencias y autenticación, ofreciendo una API REST consumida por una aplicación frontend.

El stack tecnológico actual del proyecto es:

Backend: Java 17, Spring Boot 3.2, Spring Security con JWT, Maven.

Frontend: Vue 3, Vite, Pinia, Vue Router, Axios.

Base de datos: PostgreSQL 16 (orquestrada con Docker Compose).

Herramientas de calidad: JaCoCo (cobertura), Checkstyle, PMD, ESLint.

1.2 Objetivos del Hito 4

El Hito 4 corresponde al segundo avance del trabajo de campo y está centrado en el aseguramiento de calidad mediante pruebas de software. Sus objetivos concretos son:

Elaborar un Plan de Pruebas formal para el proyecto.

Diseñar y documentar al menos 15 casos de prueba aplicando técnicas de caja negra (partición de equivalencia, valores límite, tabla de decisión, máquina de estados).

Implementar tests automatizados con un framework adecuado y dejarlos corriendo verde.

Alcanzar una cobertura $\geq 60\%$ sobre los módulos principales del sistema, medida con JaCoCo.

Documentar al menos 5 defectos detectados durante el proceso, con su severidad, estado y plan de corrección.

Sostener un historial de commits regular y descriptivo a lo largo del avance.

Aplicar Test-Driven Development (TDD) sobre al menos una funcionalidad nueva.

Elaborar un análisis FMEA simplificado sobre una funcionalidad del sistema con sus mitigaciones correspondientes.

1.3 Alcance del Documento

Este documento consolida en un único entregable la totalidad de la documentación correspondiente al Hito 4. Cubre el Plan de Pruebas, el diseño detallado de los casos de prueba con sus técnicas asociadas, los resultados de la ejecución automatizada, el análisis de cobertura, la aplicación del ciclo TDD, el reporte de defectos, el análisis de riesgos por FMEA y las pruebas basadas en estados. Las secciones están autocontenidas y siguen el orden lógico de un proceso de testing real: planificación, diseño, ejecución, análisis y conclusiones.

2. Plan de Pruebas

Esta sección define formalmente el enfoque de pruebas aplicado durante el Hito 4: qué se va a probar, con qué estrategia, con qué herramientas, bajo qué criterios y en qué tiempos.

2.1 Alcance

El Plan de Pruebas del Hito 4 abarca los siguientes módulos del sistema:

Módulo de Usuarios: modelo User y endpoints REST del UserController (creación, consulta, modificación, eliminación).

Módulo de Actividades: modelo Activity y endpoints REST del ActivityController.

Módulo de Asistencias: modelo Attendance, DTO AttendanceRequest y endpoint de registro del AttendanceController.

Manejador global de excepciones (GlobalExceptionHandler) que traduce errores de validación y conflictos de integridad a respuestas HTTP coherentes.

Capa de servicios asociada a los módulos anteriores (UserService, ActivityService, AttendanceService).

Quedan explícitamente fuera del alcance del Hito 4: el frontend Vue (el avance corresponde a un hito posterior), las clases de configuración de seguridad e inicialización (SecurityConfig, AppConfig, DataInitializer) por ser infraestructura de arranque, y los endpoints del AuthController más allá de pruebas de humo, ya que la mecánica de JWT excede el alcance del avance actual.

2.2 Estrategia de Pruebas

La estrategia adoptada combina pruebas de caja negra basadas en especificación con verificación de cobertura estructural mediante JaCoCo. La estrategia se apoya en cuatro pilares:

Diseño dirigido por técnicas formales: cada caso de prueba se justifica con una técnica explícita (partición de equivalencia, valores límite, tabla de decisión o máquina de estados).

Automatización completa: la totalidad de los casos diseñados se implementa como tests automatizados ejecutables con un único comando (mvn test).

Aplicación parcial de TDD: al menos una funcionalidad nueva se desarrolla siguiendo el ciclo Red-Green-Refactor, con evidencia del estado rojo previo al fix.

Trazabilidad: cada defecto detectado se vincula al caso de prueba que lo expuso y al commit que lo corrigió.

2.3 Niveles y Tipos de Prueba

Durante el Hito 4 se ejercen los siguientes niveles y tipos de prueba sobre el backend:

Nivel / Tipo	Qué prueba	Herramientas
Pruebas unitarias de servicio	Lógica de negocio de UserService, ActivityService y AttendanceService aislada con dobles de prueba.	JUnit 5 + Mockito
Pruebas de capa web (slice)	Validaciones de entrada y manejo de errores en los controllers mediante MockMvc, sin levantar el contexto completo de Spring.	Spring @WebMvcTest + MockMvc + Mockito
Pruebas funcionales de caja negra	Comportamiento observable a partir de la especificación: validaciones, conflictos y rutas de excepción.	JUnit 5 (mismas que arriba)
Pruebas exploratorias	Sesiones manuales no guionadas sobre el sistema desplegado para detectar comportamientos inesperados.	Navegador + Postman

2.4 Herramientas Utilizadas

Herramienta	Propósito	Versión
JUnit 5 (Jupiter)	Framework de testing automatizado.	5.x (gestionado por Spring Boot 3.2)
Mockito	Generación de dobles de prueba (mocks y stubs).	5.x
Spring Test (MockMvc)	Ejecución de pruebas de capa web sin servidor real.	6.x
JaCoCo	Medición de cobertura de código por líneas y ramas.	0.8.11
Checkstyle	Análisis estático de estilo de código.	10.x
PMD	Análisis estático de complejidad ciclomática y code smells.	6.55.0 (plugin 3.21.2)
Maven	Gestión de dependencias y ejecución de la pipeline de pruebas.	3.x
Postman	Pruebas manuales y exploratorias de la API REST.	—

Sobre la elección de JUnit y Mockito: el Plan SQA aprobado en el Hito 3 mencionaba originalmente otros frameworks como referencia, pero la consigna del Hito 4 es agnóstica al lenguaje (admite pytest, Jest u otros frameworks equivalentes). Dado que el proyecto está implementado en Java + Spring Boot, el framework natural y de uso industrial estándar es JUnit 5 junto con Mockito, decisión que se sostiene a lo largo del documento.

2.5 Criterios de Aceptación

El avance del Hito 4 se considerará aceptable si y sólo si se cumplen los siguientes criterios, todos verificables objetivamente:

Criterio	Métrica objetivo	Verificación
Casos de prueba diseñados	≥ 15 casos documentados	Sección 3 del documento
Cobertura de líneas en módulos principales	≥ 60%	Reporte JaCoCo (sección 4.3)

Tests automatizados pasando	100% (0 fallos, 0 errores)	Salida de mvn test
Defectos documentados	≥ 5 defectos con severidad y estado	Sección 6
Aplicación de TDD	≥ 1 funcionalidad nueva con ciclo Red-Green evidenciado	Sección 5
Análisis de riesgos	FMEA simplificado con ≥ 3 mitigaciones	Sección 8
Commits regulares	Historial granular y descriptivo desde el inicio del hito	git log (sección 7)

2.6 Cronograma

La planificación operativa de la semana del Hito 4 se organizó en torno a tres líneas de trabajo paralelas: el desarrollo y testing en el repositorio, la documentación y la integración de los aportes individuales del equipo. El cronograma efectivo fue:

Fecha	Actividad	Responsable
Mié 27/05	Diagnóstico y destrabe del build (3 commits: dependencia, validaciones, manejador de excepciones).	Mateo Brizuela
Mié 27/05	Diseño de los 20 casos de prueba sobre validaciones de User.	Mateo Brizuela
Jue 28/05	Implementación de tests de validación del UserController + tests del GlobalExceptionHandler.	Mateo Brizuela
Jue 28/05	Aplicación de TDD sobre Activity y Attendance (ciclos Red-Green).	Mateo Brizuela
Jue 28/05	Tests de endpoints restantes (UserController completo, Activity, Attendance, Auth happy-path).	Mateo Brizuela
Jue 28/05	Cobertura de la rama faltante en UserService.updateUser.	Mateo Brizuela
Vie 29/05 — Dom 31/05	Testing exploratorio sobre el sistema desplegado.	Ernesto
Vie 29/05 — Dom 31/05	Diseño de la máquina de estados de asistencia.	Martín Mousist
Vie 29/05 — Dom 31/05	Elaboración del análisis FMEA con mitigaciones.	Fabrizio
Lun 01/06 — Mar 02/06	Integración de aportes, pulido del documento, ensayo de presentación.	Equipo completo
Mié 03/06	Entrega y defensa.	Equipo completo

2.7 Gestión de Defectos

Los defectos detectados durante el avance se registran formalmente en la sección 6 del presente documento. Cada defecto incluye un identificador único, descripción, pasos de reproducción (o el caso de prueba que lo expuso), severidad, estado y plan de corrección. La severidad se clasifica en tres niveles:

Alta: defecto que rompe funcionalidad esencial o expone una vulnerabilidad de validación.

Media: defecto que produce un comportamiento subóptimo pero no bloqueante.

Baja: defecto cosmético o mejora menor sin impacto funcional.

Los estados posibles de un defecto son: Abierto, En análisis, Corregido y Aceptado (cuando se decide deliberadamente no corregirlo dentro del alcance del hito y se documenta la justificación).

2.8 Análisis de Riesgos

El análisis formal de riesgos se desarrolla en la sección 8 del documento mediante la técnica FMEA (Failure Mode and Effects Analysis) aplicada sobre una funcionalidad crítica del sistema, identificando modos de falla, sus efectos, causas y mitigaciones priorizadas por RPN (Risk Priority Number).

3. Diseño de Casos de Prueba

Esta sección presenta el diseño completo de los casos de prueba elaborados durante el Hito 4. Se diseñaron y documentaron 36 casos de prueba aplicando técnicas formales de caja negra sobre las funcionalidades de creación de Usuario, Actividad y Asistencia. La cantidad supera holgadamente el mínimo de 15 exigido por la consigna.

3.1 Técnicas Aplicadas

Las cuatro técnicas de caja negra se aplicaron de la siguiente manera:

Técnica	Cuándo se aplica	Dónde se ve en este documento
Partición de Equivalencia (PE)	Cuando el dominio de entrada admite ser dividido en clases que el sistema trata uniformemente.	Secciones 3.2, 3.3, 3.4
Valores Límite (VL)	Cuando una validación establece un rango numérico o de longitud (los bugs se concentran en los bordes).	Secciones 3.2, 3.3, 3.4
Tabla de Decisión	Cuando el resultado depende de combinaciones de varias condiciones.	Sección 3.5
Máquina de Estados	Cuando una entidad atraviesa estados discretos y solo ciertas transiciones son válidas.	Sección 3.6 (aporte de Martín)

La nomenclatura de identificadores sigue un esquema claro: CP-## para casos sobre Usuario, AC-## para Activity, AT-## para Attendance, EX-## para casos que ejercitan el manejador de excepciones, y TD-## para casos derivados de la tabla de decisión.

3.1.1 Clasificación por Caja Blanca y Caja Negra

Las pruebas del Hito 4 combinan las dos técnicas estructurales clásicas de la disciplina. Las pruebas de CAJA NEGRA verifican el comportamiento observable del sistema a partir de su especificación, sin conocimiento del código interno: son las pruebas de la capa web (controllers) ejecutadas vía HTTP/MockMvc, que comprueban validaciones, conflictos y rutas de excepción. Las pruebas de CAJA BLANCA, en cambio, se diseñan con conocimiento del código fuente y apuntan a ejercitar sus caminos internos (sentencias y ramas): son las pruebas unitarias de la capa de servicio (UserService, ActivityService, AttendanceService), heredadas del Hito 3 y ampliadas en este avance, que mockean los repositorios y recorren la lógica de negocio método a método.

La siguiente tabla reclasifica las 7 clases de test según la técnica de caja a la que pertenecen:

Clase de Test	Tipo de caja	Técnica principal	Qué verifica	# Tests
UserControllerTest	Caja negra	PE, VL, Tabla de decisión	Validaciones de User vía HTTP (especificación)	27
ActivityControllerTest	Caja negra	PE, VL	Validaciones de Activity vía HTTP	14
AttendanceControllerTest	Caja negra	PE, VL	Validaciones de Attendance vía HTTP	8
AuthControllerTest	Caja negra	PE	Happy-path de login (humo)	1
UserServiceTest	Caja blanca	Cobertura de sentencias y ramas	Lógica interna de UserService (incl. rama password de updateUser)	9
ActivityServiceTest	Caja blanca	Cobertura de sentencias y ramas	Lógica interna de ActivityService	7
AttendanceServiceTest	Caja blanca	Cobertura de sentencias	Lógica interna de AttendanceService	6
TOTAL	50 negra / 22 blanca	—	Mezcla de ambas técnicas	72

De este modo el avance supera el mínimo de 15 casos exigido mezclando ambas técnicas: 50 pruebas de caja negra sobre la capa web y 22 pruebas de caja blanca sobre la capa de servicio.

3.2 Casos sobre Validaciones de User

El modelo User incorpora las siguientes anotaciones de validación en sus campos:

name: @NotBlank + @Size(min=3, max=100)

email: @NotBlank + @Email

dni: @NotBlank + @Pattern(regexp = "\\d{7,8}\$")

password: @NotBlank + @Size(min=6)

role: @NotBlank + @Pattern(regexp = "^(ADMIN|TRAINER|CLIENT)\$")

Sobre estas validaciones se diseñaron 20 casos aplicando PE y VL, organizados por campo:

3.2.1 Campo dni — Partición de Equivalencia y Valores Límite

El regex `^\d{7,8}$` define una clase válida estricta: exactamente 7 u 8 dígitos numéricos. Los bordes son 6 (justo abajo del mínimo), 7 y 8 (válidos en el borde), y 9 (justo arriba del máximo).

ID	Técnica	Clase / qué prueba	Entrada	Esperado
CP-01	VL	Borde inferior válido (7 dígitos)	"1234567"	HTTP 200
CP-02	VL	Borde superior válido (8 dígitos)	"12345678"	HTTP 200
CP-03	VL	Justo abajo del mínimo (6 dígitos)	"123456"	HTTP 400
CP-04	VL	Justo arriba del máximo (9 dígitos)	"123456789"	HTTP 400
CP-05	PE	Inválida: contiene caracteres no numéricos	"1234ABC"	HTTP 400
CP-06	PE	Inválida: cadena vacía (viola @NotBlank)	""	HTTP 400

3.2.2 Campo password — Partición de Equivalencia y Valores Límite

La validación `@Size(min=6)` establece un borde inferior estricto. El campo no tiene tope superior declarado, hecho que se reporta como defecto en la sección 6.

ID	Técnica	Clase / qué prueba	Entrada	Esperado
CP-07	VL	Justo debajo del mínimo (5 caracteres)	"12345"	HTTP 400
CP-08	VL	Borde inferior válido (6 caracteres)	"123456"	HTTP 200
CP-09	PE	Válida típica	"miClave123"	HTTP 200
CP-10	PE	Inválida: vacía (viola @NotBlank)	""	HTTP 400

3.2.3 Campo name — Partición de Equivalencia y Valores Límite

La validación `@Size(min=3, max=100)` define un rango cerrado con dos bordes. Se cubre cada borde con un valor justo válido y otro justo inválido.

ID	Técnica	Clase / qué prueba	Entrada	Esperado
CP-11	VL	Justo debajo del mínimo (2 caracteres)	"ab"	HTTP 400
CP-12	VL	Borde inferior válido (3 caracteres)	"abc"	HTTP 200
CP-13	VL	Borde superior válido (100 caracteres)	"a" × 100	HTTP 200
CP-14	VL	Justo arriba del máximo (101 caracteres)	"a" × 101	HTTP 400
CP-15	PE	Inválida: vacío (viola @NotBlank)	""	HTTP 400

3.2.4 Campo role — Partición de Equivalencia

El campo rol acepta exactamente tres valores predefinidos (ADMIN, TRAINER, CLIENT). Al tratarse de un conjunto discreto cerrado, no aplica el análisis de valores límite, sólo partición de equivalencia. Es relevante notar que la validación es estrictamente case-sensitive.

ID	Técnica	Clase / qué prueba	Entrada	Esperado
CP-16	PE	Válida: dentro del conjunto permitido	"ADMIN"	HTTP 200
CP-17	PE	Inválida: fuera del conjunto	"USER"	HTTP 400
CP-18	PE	Inválida: case-sensitive (minúsculas)	"admin"	HTTP 400

3.2.5 Campo email — Partición de Equivalencia

La anotación @Email delega la validación a la implementación estándar de Jakarta Validation. Se aplica partición de equivalencia tomando un representante de la clase válida y uno de la inválida.

ID	Técnica	Clase / qué prueba	Entrada	Esperado
CP-19	PE	Válida: formato canónico	"test@mail.com"	HTTP 200
CP-20	PE	Inválida: sin arroba	"testmail.com"	HTTP 400

3.2.6 Casos sobre el GlobalExceptionHandler (EX)

Tres casos adicionales ejercitan las ramas no triviales del manejador global de excepciones, garantizando que las excepciones lanzadas por la capa de servicio se traduzcan correctamente al protocolo HTTP.

ID	Disparador	Esperado
EX-01	El service lanza DataIntegrityViolationException con mensaje que contiene "email".	HTTP 409 + body indicando email duplicado
EX-02	El service lanza DataIntegrityViolationException con mensaje que contiene "dni".	HTTP 409 + body indicando dni duplicado
EX-03	El service lanza una RuntimeException genérica.	HTTP 500

3.3 Casos sobre Validaciones de Activity

Las validaciones del modelo Activity fueron incorporadas durante el Hito 4 mediante un ciclo TDD (detallado en la sección 5). Las anotaciones agregadas son:

name: @NotBlank + @Size(min=3, max=100)

day: @NotBlank

schedule: @NotBlank

monthlyCost: @NotNull + @Positive

Sobre estas validaciones se diseñaron 10 casos. Para todos ellos se construye un objeto Activity base válido (name="Spinning", day="Lunes", schedule="18:00-19:00", monthlyCost=5000.0, active=true) y se altera únicamente el campo bajo prueba.

ID	Campo	Técnica	Clase / qué prueba	Entrada	Esperado
AC-01	monthlyCost	VL	Borde: el cero no es positivo	0.0	HTTP 400
AC-02	monthlyCost	VL	Justo arriba del cero	0.01	HTTP 200
AC-03	monthlyCost	PE	Inválida: valor negativo	-10.0	HTTP 400
AC-04	monthlyCost	PE	Inválida: nulo	null	HTTP 400
AC-05	name	VL	Justo abajo del mínimo (2 chars)	"ab"	HTTP 400
AC-06	name	VL	Borde inferior válido (3 chars)	"abc"	HTTP 200
AC-07	name	PE	Inválida: vacío	""	HTTP 400
AC-08	day	PE	Inválida: vacío	""	HTTP 400
AC-09	schedule	PE	Inválida: vacío	""	HTTP 400
AC-10	(todos)	PE	Caso feliz: actividad totalmente válida	—	HTTP 200

3.4 Casos sobre Validaciones de Attendance

El registro de asistencia se realiza mediante el DTO AttendanceRequest. Las validaciones incorporadas durante el Hito 4 sobre este DTO son:

userId: @NotNull + @Positive

notes: sin validaciones (campo opcional)

Se diseñaron 6 casos cubriendo el campo obligatorio con valores límite e inválidos, más casos válidos que confirman que notes es opcional.

ID	Campo	Técnica	Clase / qué prueba	Entrada	Esperado
AT-01	userId	PE	Inválida: nulo	null	HTTP 400
AT-02	userId	PE	Inválida: negativo	-1	HTTP 400
AT-03	userId	VL	Borde: cero no es positivo	0	HTTP 400
AT-04	userId	VL	Borde inferior válido	1	HTTP 200

AT-05	notes	PE	Válido: notes nulo (campo opcional)	null	HTTP 200
AT-06	notes	PE	Válido: notes vacío (campo opcional)	""	HTTP 200

3.5 Tabla de Decisión — Creación de Usuario

La creación de un usuario depende de la combinación de tres condiciones binarias, validadas en momentos distintos del flujo de la aplicación:

Que el email sea único en la base de datos (verificable únicamente al intentar persistir).

Que el dni sea único en la base de datos (idem).

Que el role corresponda a uno de los valores permitidos (verificable en la capa web vía @Valid, antes de tocar la base).

Como las tres condiciones son binarias, existen $2^3 = 8$ combinaciones posibles. La tabla de decisión completa es:

Regla	Email único	DNI único	Role válido	Resultado esperado	Justificación
R1	Sí	Sí	Sí	HTTP 201 Created	Camino feliz, todas las condiciones se cumplen.
R2	Sí	Sí	No	HTTP 400 Bad Request	@Valid rechaza antes de tocar la base.
R3	Sí	No	Sí	HTTP 409 Conflict (dni)	La base detecta la violación del unique sobre dni.
R4	Sí	No	No	HTTP 400 Bad Request	@Valid corta primero; no llega a la base.
R5	No	Sí	Sí	HTTP 409 Conflict (email)	La base detecta la violación del unique sobre email.
R6	No	Sí	No	HTTP 400 Bad Request	@Valid corta primero.
R7	No	No	Sí	HTTP 409 Conflict (uno de los dos)	La base lanza la primera excepción que detecta.
R8	No	No	No	HTTP 400 Bad Request	@Valid corta primero.

La lectura clave de la tabla es que cuando el campo role es inválido, las condiciones sobre email y dni se vuelven indiferentes (don't care): el flujo de @Valid corta la ejecución antes de que se intente la persistencia. Esta observación es relevante para el diseño porque demuestra que la validación en capa web actúa como un filtro previo eficiente.

De las 8 reglas, 4 casos de prueba estratégicamente elegidos cubren todas las combinaciones de respuesta:

ID	Reglas cubiertas	Setup	Esperado	Cobertura en el código
TD-01	R1	User válido, mock createUser devuelve el user.	HTTP 201	Cubierto por CP-10 (caso feliz)
TD-02	R2, R4, R6, R8	role = "USER".	HTTP 400	Cubierto por CP-17
TD-03	R3, R7	Mock service lanza DataIntegrityViolationException con "dni".	HTTP 409	Cubierto por EX-02
TD-04	R5	Mock service lanza DataIntegrityViolationException con "email".	HTTP 409	Cubierto por EX-01

Estos 4 casos derivados de la tabla de decisión ya están implementados como parte de UserControllerTest, sin necesidad de tests adicionales. La tabla de decisión cumple aquí un rol de documentación de diseño: hace explícito que la cobertura existente es deliberada y completa respecto a la matriz de condiciones.

3.6 Pruebas Basadas en Estados

El diseño detallado de la máquina de estados sobre la entidad Asistencia se desarrolla en la sección 9 del presente documento, junto con las transiciones válidas e inválidas y los casos de prueba derivados.

3.7 Resumen de Cobertura por Técnica

La distribución de los casos de prueba diseñados por técnica queda como sigue:

Técnica	Cantidad de casos	Módulos cubiertos
Partición de Equivalencia	21	User (10), Activity (5), Attendance (4), Excepciones (3, indirectamente)
Valores Límite	12	User (8), Activity (3), Attendance (1)
Tabla de Decisión	4 casos derivados (8 reglas)	User — creación
Máquina de Estados	(definidos en sección 9)	Attendance
TOTAL diseñados	≥ 36 casos	Supera el mínimo de 15 exigido por la consigna.

4. Ejecución y Resultados de los Tests Automatizados

Esta sección presenta los resultados objetivos de la ejecución del conjunto completo de pruebas automatizadas al cierre del Hito 4, junto con el análisis de cobertura asociado.

4.1 Setup Técnico de los Tests

La capa de testing se estructuró en dos niveles claramente diferenciados:

4.1.1 Tests Unitarios de la Capa de Servicio

Las clases `ActivityServiceTest`, `AttendanceServiceTest` y `UserServiceTest` verifican la lógica de negocio en aislamiento total. Utilizan Mockito puro para mockear los repositorios JPA y construyen las entidades en memoria. Estos tests son rápidos (orden de milisegundos por test) y no levantan ningún contexto de Spring.

4.1.2 Tests de Capa Web (Slice)

Las clases `UserControllerTest`, `ActivityControllerTest`, `AttendanceControllerTest` y `AuthControllerTest` verifican el comportamiento del sistema desde el punto de vista de la API HTTP, incluyendo la activación de `@Valid`, la serialización JSON y el manejo global de excepciones. Para ello se utiliza la combinación de anotaciones `@WebMvcTest`, `@AutoConfigureMockMvc(addFilters = false)`, `@Import(GlobalExceptionHandler.class)` y `@MockBean` sobre las dependencias requeridas.

Un detalle técnico relevante surgido durante la implementación: para que `@WebMvcTest` funcionara con un proyecto que tiene Spring Security activo, fue necesario: Desactivar los filtros de seguridad en el slice (`addFilters = false`), de modo que las requests no fueran rechazadas con 401/403 antes de llegar a las validaciones. Mockear como `@MockBean` a `JwtUtil`, `UserDetailsService` (la interfaz, no la implementación) y `PasswordEncoder`, para que el contexto Spring de prueba se levantara sin errores de beans faltantes.

Importar explícitamente `GlobalExceptionHandler` vía `@Import`, ya que `@WebMvcTest` no lo descubre automáticamente por defecto.

En el caso particular de `AuthControllerTest`, se agregaron `@MockBean` adicionales sobre `AuthenticationManager` y `UserRepository`, dado que `AuthController` los inyecta directamente.

4.2 Resultados de Ejecución

La ejecución completa de la suite de tests al cierre del Hito 4 arrojó el siguiente resultado:

Métrica	Valor
Total de tests ejecutados	72
Tests exitosos (passed)	72
Tests fallidos (failed)	0

Tests con error (errors)	0
Tests saltados (skipped)	0
Resultado global de mvn clean test	BUILD SUCCESS

Comparación con el estado al inicio del hito:

Fase	Tests totales	Tests fallidos
Inicio del Hito 4 (heredados del Hito 3)	21	0
Cierre del Hito 4	72	0
Incremento neto	+51	—

Distribución por clase de test:

Clase de Test	Cantidad	Tipo
UserServiceTest	9	Unitario (servicio)
ActivityServiceTest	7	Unitario (servicio)
AttendanceServiceTest	6	Unitario (servicio)
UserControllerTest	27	Capa web (slice)
ActivityControllerTest	14	Capa web (slice)
AttendanceControllerTest	8	Capa web (slice)
AuthControllerTest	1	Capa web (slice)

4.3 Cobertura de Código (JaCoCo)

La medición de cobertura se realizó con JaCoCo 0.8.11 invocado mediante mvn jacoco:report. El reporte final por paquete es el siguiente:

Paquete	Líneas cubiertas	% líneas	Ramas cubiertas	% ramas
controller/	25/25	100%	—	—
service/	41/41	100%	4/4	100%
model/	28/28	100%	(Lombok*)	—
dto/	11/12	91.7%	(Lombok*)	—
config/ (incluye GlobalExceptionHandler)	24/93	25.8%	5/12	41.7%

security/	1/45	2.2%	0/14	0%
TOTAL	131/247	53.0%	10/258	3.9%

(*) Las ramas de los paquetes model/ y dto/ corresponden a código generado por Lombok (equals, hashCode, toString, builders). No representan lógica de negocio y, por convención de la industria, no se exigen cubrir.

Cobertura específica del GlobalExceptionHandler dentro del paquete config/:

Método	Cobertura
handleValidationException	8/8 líneas (100%)
handleDataIntegrityViolation	9/9 líneas (100%)
handleRuntimeException	3/3 líneas (100%)
Lambdas internas (mapeo de errores)	3/3 líneas (100%)
Total GlobalExceptionHandler	23/23 líneas (100%)

4.3.1 Cobertura de Ramas y Complejidad Ciclomática

Las pruebas de caja blanca de la capa de servicio se evalúan no sólo por líneas cubiertas sino por su cobertura de ramas (branch coverage) y por la complejidad ciclomática (CC) de los métodos que ejercitan. La cobertura de ramas mide qué porcentaje de las decisiones (if/else, &&, ||) fue recorrido por al menos un test; la complejidad ciclomática cuenta los caminos de ejecución independientes de cada método y es el límite teórico de casos necesarios para cubrirlo.

Cobertura de ramas (JaCoCo). El paquete service/ alcanza 4/4 ramas cubiertas = 100%. La única rama que faltaba al inicio del hito era el segundo operando de la condición compuesta de UserService.updateUser (password no-nula pero en blanco); el defecto DEF-03 documenta su detección y el test que la cerró, llevando el branch coverage del servicio al 100%.

Complejidad ciclomática (PMD 6.55.0, regla CyclomaticComplexity con methodReportLevel=10). La ejecución de mvn pmd:check arrojó 0 violaciones: ningún método del proyecto supera CC = 10, el umbral fijado en el Plan SQA. Los métodos de negocio más complejos tienen una complejidad baja, lo que explica que un número reducido de casos blancos alcance el 100% de cobertura de ramas:

Método (service/)	Complejidad ciclomática	Ramas	Cobertura de ramas
UserService.updateUser	3	4	4/4 (100%)
ActivityService.updateActivity	2	2	100%
AttendanceService.registerAttendance	2	—	100%
Umbral Plan SQA	CC ≤ 10	—	CUMPLIDO (0 violaciones PMD)

Evidencia: docs/evidencia/05_pmd_output.md (salida de PMD) y el reporte JaCoCo (ramas del paquete service/).

4.4 Análisis del Porcentaje de Cobertura

El número global de 53.0% requiere lectura contextualizada para no confundir la métrica con su interpretación. El criterio de la consigna establece $\geq 60\%$ sobre módulos principales. Los módulos principales son los que contienen lógica de negocio del sistema:

Módulo principal	Cobertura	¿Cumple $\geq 60\%$?
controller/	100%	Sí
service/	100%	Sí
model/	100%	Sí
dto/	91.7%	Sí
GlobalExceptionHandler	100%	Sí

La caída del porcentaje global a 53% se explica íntegramente por los paquetes security/ y por la parte de config/ que contiene SecurityConfig, AppConfig y DataInitializer: security/: contiene los componentes de Spring Security (JwtAuthenticationFilter, JwtUtil, UserDetailsServiceImpl). Su testing unitario requeriría montaje de infraestructura de autenticación y excede el alcance del Hito 4, que está focalizado en pruebas funcionales de caja negra.

config/ (sin contar GlobalExceptionHandler): SecurityConfig configura la cadena de filtros de seguridad; AppConfig declara beans de infraestructura; DataInitializer puebla la base con datos de bootstrap. Ninguna de las tres clases contiene lógica de negocio testeable por unit testing — son configuración de aplicación.

Esta exclusión deliberada está alineada con la cláusula de cobertura sobre código nuevo y módulos principales del Plan SQA aprobado en el Hito 3, y con la práctica industrial estándar de no exigir cobertura unitaria sobre clases de configuración Spring.

Conclusión sobre cobertura Sobre módulos principales (controller, service, model, dto, GlobalExceptionHandler), la cobertura efectiva es prácticamente 100%. El criterio de aceptación $\geq 60\%$ se supera holgadamente. El código nuevo introducido durante el Hito 4 (validaciones de User/Activity/Attendance, GlobalExceptionHandler y @Valid en controllers) está cubierto al 100% por tests automatizados.

4.5 Integración Continua (Pipeline de GitHub Actions)

La ejecución de los tests se automatizó en un pipeline de Integración Continua definido en .github/workflows/ci.yml. El workflow se dispara en cada git push y en cada pull request sobre cualquier rama, de modo que la suite ya no depende de la ejecución manual local.

En cada ejecución el pipeline: (1) prepara el entorno con Java 17 (Temurin) y caché de dependencias Maven; (2) compila y corre la totalidad de los tests con mvn verify, generando el reporte de cobertura JaCoCo; (3) publica el reporte JaCoCo como artefacto descargable; (4) en los pull requests comenta automáticamente el porcentaje de cobertura y lo contrasta contra un umbral mínimo del 60%; y (5) ejecuta los análisis estáticos Checkstyle y PMD, publicando el reporte de PMD como artefacto.

Con esto el avance cumple el requisito de tests automatizados dentro de un pipeline de GitHub Actions con ejecución automatizada y verificación de cobertura mínima.

5. Aplicación de TDD (Test-Driven Development)

Esta sección documenta la aplicación del ciclo de Test-Driven Development sobre dos funcionalidades del sistema durante el Hito 4, en cumplimiento de la tarea solicitada por la cátedra.

5.1 Concepto y Ciclo Red-Green-Refactor

Test-Driven Development (TDD) es una metodología de desarrollo de software que invierte el orden tradicional de la construcción de funcionalidad. En lugar de programar primero y luego verificar con tests, en TDD el flujo es:

RED (Rojo): se escribe un test que describe el comportamiento esperado. El test se ejecuta y falla, porque la funcionalidad todavía no existe.

GREEN (Verde): se implementa el mínimo código de producción necesario para que el test pase. Nada más.

REFACTOR: con la red de seguridad del test verde, se mejora la estructura del código sin alterar su comportamiento.

Los beneficios de TDD son varios: el diseño se conduce desde el caso de uso, no se escribe código sin razón, los tests sirven como documentación viva y la cobertura es automáticamente alta. El ciclo se aplicó sobre dos funcionalidades nuevas introducidas en este hito: las validaciones de entrada de `ActivityController` y de `AttendanceController`.

5.2 Aplicación sobre `ActivityController`

5.2.1 Estado inicial

Al comienzo del Hito 4, `ActivityController` no aplicaba `@Valid` sobre el cuerpo de las requests y el modelo `Activity` no tenía anotaciones de validación. Esto permitía que un cliente enviara una actividad con campos vacíos, costo negativo o costo nulo y obtuviera, en el mejor de los casos, un error 500 difuso (en el peor, datos basura persistentes).

5.2.2 Fase RED — Tests previos al fix

Se creó la clase `ActivityControllerTest` con los 10 casos diseñados (AC-01 a AC-10) y se ejecutó la suite. Resultado:

Resultado	Cantidad	Detalle
Tests que fallaron (rojo esperado)	7	AC-01, AC-03, AC-04, AC-05, AC-07, AC-08, AC-09
Tests que pasaron	3	AC-02, AC-06, AC-10 (casos válidos)

La razón del fallo fue uniforme en los 7 casos: el test esperaba un HTTP 400, pero el sistema respondía HTTP 200 porque dejaba pasar la entrada inválida. Esta evidencia documental del estado rojo se guardó en el repositorio bajo docs/evidencia/hito4/01_tdd_activity_red.md.

Commit asociado al estado rojo:

Hash	Mensaje
4d21072	test(activity): add validation tests for ActivityController (TDD red)

5.2.3 Fase GREEN — Implementación de las validaciones

Se agregaron las anotaciones de validación al modelo Activity:

name: @NotBlank + @Size(min=3, max=100)

day: @NotBlank

schedule: @NotBlank

monthlyCost: @NotNull + @Positive

Y se agregó @Valid en los métodos createActivity y updateActivity del controller. Se re-ejecutó la suite y los 10 tests pasaron sin modificarse:

Resultado	Cantidad
Tests pasados tras el fix	10/10
Tests modificados	0
Suite global	BUILD SUCCESS

Commit asociado al estado verde:

Hash	Mensaje
a9c721e	feat(activity): add validation constraints and @Valid (TDD green)

5.3 Aplicación sobre AttendanceController

Se aplicó el mismo ciclo Red-Green sobre AttendanceController, esta vez orientado al DTO AttendanceRequest, que no tenía validaciones.

5.3.1 Fase RED

Se creó AttendanceControllerTest con los 6 casos AT-01 a AT-06. Resultado de la primera ejecución:

Resultado	Cantidad	Detalle
-----------	----------	---------

Tests que fallaron (rojo esperado)	3	AT-01, AT-02, AT-03
Tests que pasaron	3	AT-04, AT-05, AT-06 (casos válidos)

Commit del estado rojo: 9bbddc8 — test(attendance): add validation tests for AttendanceController (TDD red).

5.3.2 Fase GREEN

Se agregaron las validaciones @NotNull y @Positive al campo userId del DTO AttendanceRequest, y @Valid en el método registerAttendance del controller. La re-ejecución dio los 6 tests en verde.

Commit del estado verde: 21b0416 — feat(attendance): add validation to AttendanceRequest and @Valid (TDD green).

5.4 Evidencia y Conclusiones del Ciclo TDD

La aplicación de TDD cumplió simultáneamente con tres objetivos del Hito 4 en un único bloque coherente de trabajo:

Tarea del profesor: ciclo Red-Green documentado sobre dos funcionalidades nuevas con commits separados que dejan el ciclo trazable en el historial de git.

Detección de defectos: los tests en rojo expusieron y documentaron formalmente los defectos 1 y 2 del reporte (sección 6).

Aumento de cobertura: las nuevas funcionalidades quedaron 100% cubiertas en el momento mismo de su nacimiento.

La evidencia detallada de cada estado (capturas de las salidas de mvn test, listado de tests fallidos con sus aserciones específicas) se encuentra en los archivos 01_tdd_activity_red.md, 02_tdd_activity_green.md, 03_tdd_attendance_red.md y 04_tdd_attendance_green.md del directorio docs/evidencia/hito4 del repositorio.

6. Reporte de Defectos

Esta sección documenta los defectos detectados durante el avance del Hito 4. La consigna establece un mínimo de 5 defectos; el avance documenta 6 defectos, de los cuales 3 fueron corregidos en el mismo hito mediante TDD y testing dirigido, 1 fue mitigado mediante un test adicional, y 2 quedan registrados como Abiertos para análisis o tratamiento en avances posteriores.

6.1 Resumen Cuantitativo

Severidad	Cantidad	Corregidos	Abiertos
Alta	2	2	0
Media	3	1	2
Baja	1	1	0

TOTAL	6	4	2
-------	---	---	---

6.2 Tabla Consolidada de Defectos

ID	Descripción	Severidad	Estado
DEF-01	ActivityController no aplica @Valid sobre el cuerpo de las requests y Activity carece de anotaciones de validación.	Alta	Corregido
DEF-02	AttendanceController no aplica @Valid y AttendanceRequest carece de validaciones sobre userId.	Alta	Corregido
DEF-03	UserService.updateUser presenta una rama no cubierta en la condición compuesta que evalúa si la password es no-nula pero blank.	Media	Corregido
DEF-04	User.password tiene @Size(min=6) pero carece de tope superior, permitiendo contraseñas arbitrariamente largas.	Media	Abierto
DEF-05	La validación de User.role es estrictamente case-sensitive ("admin" en minúsculas es rechazado).	Baja	Corregido (decisión de diseño documentada)
DEF-06	LoginRequest no posee anotaciones de validación; permite enviar credenciales vacías al endpoint de autenticación.	Media	Abierto

6.2.1 Matriz de Trazabilidad y Prioridad de Defectos

La siguiente matriz completa la trazabilidad exigida para cada defecto, distinguiendo la SEVERIDAD (impacto técnico, asignada por QA) de la PRIORIDAD (urgencia de corrección, asignada por el Product Owner), e indicando quién lo reportó, a quién se asignó, dónde se detectó y cuál es su test de verificación. La versión afectada es 0.0.1-SNAPSHOT en todos los casos.

ID	Severidad (QA)	Prioridad (PO)	Reportado por	Asignado a	Detectado en	Test de verificación	Estado
DEF-01	Alta	Alta	Mateo Brizuela (QA)	Mateo Brizuela	Fase RED TDD (ActivityControllerTest)	ActivityControllerTest AC-01..AC-10 (a9c721e)	Corregido
DEF-02	Alta	Alta	Mateo Brizuela (QA)	Mateo Brizuela	Fase RED TDD (AttendanceControllerTest)	AttendanceControllerTest AT-01..AT-06 (21b0416)	Corregido

DEF-03	Media	Media	Mateo Brizuela (JaCoCo)	Mateo Brizuela	Reporte JaCoCo (rama no cubierta)	UserServiceTest blank password (e7a0766)	Corregido
DEF-04	Media	Baja	Mateo Brizuela (inspección)	Equipo (pendiente)	Modelo User.password	Pendiente (mejora próximo avance)	Abierto
DEF-05	Baja	Baja	Mateo Brizuela (CP-18)	Mateo Brizuela	UserControllerTest (CP-18)	UserControllerTest CP-18	Corregido*
DEF-06	Media	Media	Mateo Brizuela (inspección DTOs)	Equipo (próximo avance)	LoginRequest / AuthController	Pendiente (próximo avance)	Abierto
DEF-EXP-01	Menor	Media	Ernesto (exploratorio)	Equipo	POST /attendances userId inexistente	ME-05 (maquina de estados)	Abierto
DEF-EXP-02	Mayor	Alta	Ernesto (exploratorio)	Equipo	POST /attendances duplicado	ME-07 (maquina de estados)	Abierto
DEF-EXP-03	Mayor	Alta	Ernesto (exploratorio)	Equipo	PUT /users con password vacia	Pendiente	Abierto
DEF-EXP-04	Critica	Alta	Ernesto (exploratorio)	Equipo	GET /users como CLIENT (sin RBAC)	Pendiente	Abierto
DEF-EXP-05	Menor	Baja	Ernesto (exploratorio)	Equipo	DELETE /users con FK activa	Pendiente	Abierto
DEF-EXP-06	Menor	Baja	Ernesto (exploratorio)	Equipo	POST /activities day/schedule libres	Pendiente	Abierto

(*) DEF-05 se cerro como decision de diseno consciente (la validacion de role se mantiene case-sensitive). La columna Prioridad refleja la urgencia de negocio: los defectos criticos/mayores abiertos del testing exploratorio (DEF-EXP-02, 03 y 04) son los de mayor prioridad para el proximo avance, pese a no haber sido corregidos en este hito.

6.3 Detalle de Cada Defecto

DEF-01 — Validaciones ausentes en ActivityController

Campo	Valor
Descripción	El endpoint POST /api/activities aceptaba sin validar actividades con name vacío, day vacío, schedule vacío, monthlyCost nulo, negativo o cero. La falta de @Valid en el controller y de anotaciones de validación en el modelo Activity permitía que datos inválidos llegarán a la capa de servicio y eventualmente a la base de datos.
Cómo se detectó	Casos AC-01, AC-03, AC-04, AC-05, AC-07, AC-08, AC-09 del diseño de pruebas — todos fallaron en la fase RED del ciclo TDD.

Severidad	Alta — la API expone datos inválidos a la capa de persistencia, pudiendo resultar en respuestas HTTP 500 o, en presencia de constraints débiles, datos corruptos.
Plan de corrección	Agregar @Valid en createActivity y updateActivity; agregar @NotBlank, @Size, @NotNull, @Positive en Activity según corresponda a cada campo.
Estado	Corregido
Commit de corrección	a9c721e — feat(activity): add validation constraints and @Valid (TDD green)

DEF-02 — Validaciones ausentes en AttendanceController/AttendanceRequest

Campo	Valor
Descripción	El endpoint POST /api/attendances aceptaba peticiones con userId nulo, negativo o cero. El DTO AttendanceRequest no tenía anotaciones de validación y el controller no aplicaba @Valid.
Cómo se detectó	Casos AT-01, AT-02, AT-03 — fallaron en la fase RED del ciclo TDD.
Severidad	Alta — un userId inválido derivaba en consultas a la base con valores que no podían corresponder a ningún usuario real.
Plan de corrección	Agregar @NotNull + @Positive en userId de AttendanceRequest; agregar @Valid en registerAttendance.
Estado	Corregido
Commit de corrección	21b0416 — feat(attendance): add validation to AttendanceRequest and @Valid (TDD green)

DEF-03 — Rama no cubierta en UserService.updateUser

Campo	Valor
Descripción	El método UserService.updateUser contiene una condición compuesta del tipo "si password no es null y password no es blank, actualizar". El segundo operando (la verificación de blank cuando la password es no-nula) no estaba ejercitado por ningún test, dejando 1 de 4 ramas marcadas como missed por JaCoCo.
Cómo se detectó	Reporte JaCoCo durante el análisis de cobertura del Hito 4.
Severidad	Media — el código existe y funciona, pero una rama sin test es deuda técnica y un riesgo latente.
Plan de corrección	Agregar test cuandoPasswordEsBlank en UserServiceTest que cubra el caso password=" ".
Estado	Corregido
Commit de corrección	e7a0766 — test(user): cover missed branch in updateUser (blank password)

DEF-04 — User.password sin tope superior

Campo	Valor
Descripción	La anotación @Size(min=6) sobre el campo password establece un mínimo pero no un máximo. En la práctica, esto permite enviar contraseñas de longitud arbitraria al endpoint de creación de usuarios.
Cómo se detectó	Inspección de las anotaciones del modelo User durante el diseño de los casos de prueba.
Severidad	Media — vector potencial de denegación de servicio si se hashea una entrada extremadamente grande, y posible inconsistencia con políticas de seguridad razonables.
Plan de corrección	Agregar max=128 (o un valor consensuado con la política de contraseñas del proyecto) a la anotación @Size.
Estado	Abierto — se documenta como mejora para un avance posterior, ya que la corrección requiere acuerdo del equipo sobre la política de contraseñas.

DEF-05 — Validación de role estrictamente case-sensitive

Campo	Valor
Descripción	La validación @Pattern("^(ADMIN TRAINER CLIENT)\$") sobre el campo role rechaza con HTTP 400 cualquier valor que no esté en mayúsculas exactas. Una entrada como "admin" o "Admin" se rechaza.
Cómo se detectó	Caso CP-18 del diseño de pruebas.
Severidad	Baja — no es una falla funcional sino una decisión de diseño que conviene hacer explícita.
Decisión	Se considera un comportamiento deseado: forzar la normalización en el cliente evita ambigüedades en autorización. Se documenta como decisión consciente de diseño y se mantiene el comportamiento actual.
Estado	Corregido (cerrado por decisión de diseño)

DEF-06 — LoginRequest sin validaciones

Campo	Valor
Descripción	El DTO LoginRequest, utilizado por el endpoint de autenticación, no posee anotaciones de validación. Esto permite enviar peticiones con email/password vacíos al endpoint de login.
Cómo se detectó	Inspección de DTOs durante el análisis de cobertura.
Severidad	Media — no es una vulnerabilidad de seguridad directa (la autenticación fallará igual), pero genera respuestas 500 en lugar de un 400 limpio cuando faltan credenciales.
Plan de corrección	Agregar @NotBlank en email y password de LoginRequest, y @Valid en el endpoint de login del AuthController.
Estado	Abierto — queda fuera del alcance funcional del Hito 4 (que se centró en validaciones de los modelos de dominio). Se planifica para el siguiente avance.

6.4 Defectos Detectados por Testing Exploratorio

Resumen — Testing Exploratorio:

Severidad	Cantidad	Estado
Crítica	1	Abierto
Mayor	2	Abierto
Menor	3	Abierto
Total	6	—

DEF-EXP-01 — userId inexistente devuelve HTTP 500 en lugar de 404

Campo	Valor
Descripción	Al registrar asistencia con userId positivo pero inexistente en la DB, el sistema devuelve HTTP 500 en lugar de 404.
Cómo se detectó	POST /api/attendances con userId=99999.
Severidad	Menor
Plan de corrección	Crear excepción NotFoundException y mapearla a HTTP 404 en GlobalExceptionHandler.
Estado	Abierto

DEF-EXP-02 — Se permiten asistencias duplicadas sin restricción

Campo	Valor
Descripción	El sistema acepta múltiples asistencias del mismo usuario el mismo día. Ambos registros se persisten con HTTP 200.
Cómo se detectó	POST /api/attendances con userId=2 ejecutado dos veces seguidas — ambas devuelven 200.
Severidad	Mayor — altera el historial y las estadísticas de asistencia.
Plan de corrección	Validar en AttendanceService si ya existe asistencia del mismo userId en la fecha actual. Si existe, devolver HTTP 409.
Estado	Abierto

DEF-EXP-03 — No se puede editar usuario sin cambiar la contraseña

Campo	Valor
-------	-------

Descripción	Al editar un usuario dejando contraseña vacía (UI indica "dejar vacío para no cambiar"), el backend devuelve error 400. La funcionalidad de edición está rota.
Cómo se detectó	PUT /api/users/{id} con password="" — respuesta: 400 "La contraseña no puede estar vacía".
Severidad	Mayor — impide modificar datos de usuarios sin forzar cambio de contraseña.
Plan de corrección	Crear DTO UpdateUserRequest sin @NotBlank en password, o usar grupos de validación para diferenciar creación de actualización.
Estado	Abierto

DEF-EXP-04 — Sin control de roles: CLIENT accede a todas las rutas (CRÍTICO)

Campo	Valor
Descripción	Un usuario CLIENT puede crear/editar/eliminar usuarios, actividades y registrar asistencias de otros. No hay RBAC implementado.
Cómo se detectó	Login como CLIENT; GET /api/users con ese token devuelve HTTP 200 con la lista completa.
Severidad	Crítica — un cliente del gimnasio puede crear cuentas de administrador o eliminar usuarios.
Plan de corrección	Agregar autorización por rol en SecurityConfig (.hasRole("ADMIN")) o con @PreAuthorize en los controllers.
Estado	Abierto
Nota	Los tests automatizados no detectaron este defecto porque @AutoConfigureMockMvc(addFilters=false) deshabilita los filtros de seguridad.

DEF-EXP-05 — Mensaje incorrecto al eliminar usuario con asistencias

Campo	Valor
Descripción	Al eliminar un usuario con asistencias asociadas, el sistema devuelve HTTP 409 con el mensaje "Verifique que el email y DNI sean únicos", que no corresponde al error real. El frontend no muestra ningún mensaje de error.
Cómo se detectó	Registrar asistencia para un usuario, luego DELETE /api/users/{id}.
Severidad	Menor — el sistema no pierde datos pero el mensaje confunde.
Plan de corrección	Agregar detección de FK violation en GlobalExceptionHandler. En frontend, agregar try-catch en confirmDelete().
Estado	Abierto

DEF-EXP-06 — Campos día y horario de actividad sin validación de formato en backend

Campo	Valor
-------	-------

Descripción	Los campos day y schedule aceptan cualquier string en el backend. El frontend restringe day a un select, pero accediendo directo a la API se insertan datos inconsistentes.
Cómo se detectó	POST /api/activities con day="CUALQUIER DIA" y schedule="mañana temprano" — HTTP 200, se persiste.
Severidad	Menor — genera datos inconsistentes sin error visible.
Plan de corrección	Agregar @Pattern en el campo day de Activity con los 7 días válidos.
Estado	Abierto

¿Por qué el testing exploratorio encontró lo que los tests formales no encontraron?

Defecto	Por qué no se detectó antes
DEF-EXP-01	No se probó userId positivo inexistente en los casos formales
DEF-EXP-02	No se diseñó caso de doble registro
DEF-EXP-03	El flujo editar-sin-cambiar-pass no estaba en los casos formales
DEF-EXP-04	Los tests deshabilitan filtros de seguridad (addFilters=false)
DEF-EXP-05	No se diseñó caso de DELETE con FK activa
DEF-EXP-06	No se probó la API directamente sin el frontend

Esto ilustra el Principio 5 de ISTQB (Paradoja del pesticida): los tests formales encontraron los bugs que estaban diseñados para encontrar. El testing exploratorio encontró los que quedaron fuera del diseño.

7. Evolución del Repositorio durante el Hito 4

Esta sección documenta la actividad sostenida sobre el repositorio durante la semana del Hito 4, en cumplimiento del criterio de "commits regulares y descriptivos" exigido por la consigna.

7.1 Commits del Hito 4

Durante el avance se realizaron 11 commits de desarrollo y testing en la rama main, todos siguiendo la convención Conventional Commits (chore, feat, test, docs) y describiendo de manera precisa el cambio incorporado. Listado en orden cronológico ascendente:

#	Hash	Mensaje	Tipo
1	d68480e	chore(backend): add spring-boot-starter-validation dependency	chore

2	8491637	feat(user): add bean validation constraints to user model	feat
3	95d30fa	feat(backend): add global exception handler for validation errors	feat
4	f66e801	test(user): add controller validation and exception handler tests	test
5	4d21072	test(activity): add validation tests for ActivityController (TDD red)	test
6	a9c721e	feat(activity): add validation constraints and @Valid (TDD green)	feat
7	9bbddc8	test(attendance): add validation tests for AttendanceController (TDD red)	test
8	21b0416	feat(attendance): add validation to AttendanceRequest and @Valid (TDD green)	feat
9	eba3183	test(user): add tests for remaining UserController endpoints	test
10	bc6138d	test(controllers): add happy-path tests for remaining endpoints	test
11	e7a0766	test(user): cover missed branch in updateUser (blank password)	test

Adicionalmente, un duodécimo commit (1f0d8d1) consolidó la evidencia completa del hito bajo docs/evidencia/hito4/. Posteriormente se sumaron los commits de cierre del hito (evidencia FMEA, pipeline de Integración Continua, informe consolidado y presentación), totalizando 18 commits del Hito 4 en el repositorio.

7.2 Análisis del Historial

La distribución de tipos de commit refleja un proceso disciplinado:

Tipo	Cantidad	Proporción
feat (funcionalidad nueva)	4	33%
test (incorporación de tests)	7	58%
chore (mantenimiento)	1	8%
docs (documentación)	1 (consolidación final)	—

La proporción 58% de commits de tipo test refleja correctamente que el Hito 4 está centrado en testing. Los commits de tipo feat corresponden a funcionalidades nuevas introducidas siguiendo TDD: cada uno aparece inmediatamente después del commit test correspondiente, dejando la secuencia Red-Green visible en el historial.

7.3 Métricas Acumuladas

Métrica	Inicio Hito 4	Cierre Hito 4	Variación
Total de commits en main	13	31	+18
Tests automatizados	21	72	+51
Cobertura de líneas global	28.3%	53.0%	+24.7 p.p.
Cobertura controller/	0%	100%	+100 p.p.
Cobertura GlobalExceptionHandler	(no existía)	100%	—
Defectos documentados	0	6	+6

8. Análisis de Riesgos — FMEA

Esta sección presenta el análisis de modos de falla y efectos (FMEA — Failure Mode and Effects Analysis) aplicado sobre una funcionalidad crítica del sistema, junto con la priorización de mitigaciones por RPN (Risk Priority Number).

8.1 Marco Conceptual

FMEA es una técnica sistemática de análisis de riesgos que enumera, para cada componente o funcionalidad, los modos posibles de falla, sus efectos sobre el sistema y el usuario, sus causas raíces, y los controles existentes para detectarla o prevenirla. Cada modo se cuantifica con tres escalas de 1 a 10:

S (Severidad): qué tan grave es el efecto de la falla si ocurre.

P (Probabilidad de ocurrencia): qué tan probable es que la falla suceda.

D (Detectabilidad): qué tan difícil es detectar la falla antes de que llegue al usuario (10 = imposible detectar, 1 = imposible que pase desapercibida).

El RPN se calcula como el producto de los tres: $RPN = S \times P \times D$. Los modos de falla se priorizan por RPN decreciente y se diseñan mitigaciones para los de mayor puntaje.

8.2 Funcionalidad Analizada

La funcionalidad seleccionada para el análisis FMEA es "Registrar Asistencia de un usuario al gimnasio" (endpoint POST /api/attendances). Esta funcionalidad fue elegida por reunir varias características que la hacen un caso de estudio ideal:

Es transversal: cruza autenticación, validación, lógica de servicio y persistencia.

Tiene impacto directo sobre el negocio: una falla puede afectar el conteo de presencias, derivado en facturación o estadísticas.

Manipula datos sensibles del usuario (identidad y fecha).

8.3 Análisis FMEA

El análisis detallado, incluyendo la enumeración de modos de falla con sus puntajes S, P, D y RPN, así como el plan completo de mitigaciones, será desarrollado por Fabricio durante la semana previa a la entrega e integrado en esta sección antes del cierre final del documento.

8.4 Mitigaciones Priorizadas

Las tres mitigaciones de mayor prioridad derivadas del análisis FMEA, junto con su justificación, su impacto esperado sobre los puntajes S/P/D, y su responsable de implementación, se documentará en esta subsección.

8.5 FMEA del módulo "Registrar Asistencia" — Power Strike

Tabla de Modos de Falla

#	Modo de fallo	Efecto	S	P	D	RPN
1	Falla de base de datos durante el guardado de la asistencia	La asistencia no se registra y se pierde información importante sobre la concurrencia de los usuarios	8	5	8	320
2	Doble registro accidental de la misma asistencia	El usuario aparece más de una vez, alterando estadísticas e informes de asistencia	5	6	7	210
3	Error de red durante el registro	El usuario cree que la asistencia fue registrada, pero el dato no llega al servidor	7	5	6	210
4	Registro de asistencia para un usuario inexistente	Se generan datos inconsistentes o errores en los reportes del sistema	7	3	7	147
5	Registro de asistencia con fecha futura	Se almacenan asistencias inválidas que afectan estadísticas y controles de actividad	4	4	5	80

Orden de Prioridad (Mayor RPN → Menor RPN)

Mitigaciones Propuestas

MITIGACIÓN 1

Modo de fallo: Falla de base de datos durante el guardado (RPN 320).

Estrategia: Detección + Recuperación.

Implementación concreta: Registrar errores mediante logs y devolver un mensaje controlado al usuario. Implementar reintentos automáticos de conexión y monitoreo de disponibilidad de la base de datos para detectar fallas tempranamente.

MITIGACIÓN 2

Modo de fallo: Doble registro accidental de asistencia (RPN 210).

Estrategia: Programación defensiva.

Implementación concreta: Validar en el backend que no exista una asistencia previa del mismo usuario para la misma actividad en la misma fecha. Si existe, rechazar el registro y mostrar un mensaje informativo.

MITIGACIÓN 3

Modo de fallo: Error de red durante el registro (RPN 210).

Estrategia: Defensa en profundidad.

Implementación concreta: Confirmar desde el servidor que la asistencia fue almacenada correctamente antes de mostrar éxito al usuario. Si ocurre una falla de comunicación, informar el error y permitir reintentar la operación sin duplicar registros.

9. Pruebas Basadas en Estados

Diagrama de Estados General

Diagrama de Estados en función al Historial de asistencias

Esta sección documenta el diseño y la verificación de la máquina de estados sobre una entidad del sistema, junto con los casos de prueba derivados de sus transiciones válidas e inválidas.

9.1 Marco Conceptual

La técnica de pruebas basadas en estados (state-based testing) se aplica a entidades que atraviesan un conjunto discreto y finito de estados a lo largo de su ciclo de vida, en donde solo ciertas transiciones entre estados son válidas. Una máquina de estados modela explícitamente:

El conjunto de estados posibles de la entidad.

Las transiciones válidas (estado actual + evento $\Rightarrow$ estado siguiente).

Las transiciones inválidas que el sistema debe rechazar.

La técnica complementa a las pruebas de validación de entrada: mientras las técnicas de PE y VL verifican que un dato individual sea aceptable, las pruebas basadas en estados verifican que una secuencia de operaciones sea consistente con el modelo del dominio.

9.2 Entidad Analizada

Se eligió Attendance porque modela un ciclo de vida con restricciones claras. El sistema actual no implementa todas las restricciones esperadas, lo que la hace especialmente interesante. No existe PUT ni DELETE — las asistencias son inmutables una vez registradas.

9.3 Diagrama de Estados

Estados:

INEXISTENTE (inicial): no existe registro de asistencia

REGISTRADA: asistencia persistida exitosamente

RECHAZADA: request rechazado por validación (HTTP 400) — estado terminal

ERROR_SISTEMA: userId válido pero inexistente en DB (HTTP 500) — estado terminal

Transiciones:

REGISTRADA $\rightarrow$ REGISTRADA: POST mismo userId ■■■ [HTTP 200 — Bug DEF-EXP-02, debería ser 409]

Tabla de Transiciones:

Tipo	Estado origen	Evento	Estado destino	HTTP esperado
Válida	INEXISTENTE	POST userId existente	REGISTRADA	200

Válida	INEXISTENTE	POST userId = null	RECHAZADA	400
Válida	INEXISTENTE	POST userId negativo	RECHAZADA	400
Válida	INEXISTENTE	POST userId = 0 (borde)	RECHAZADA	400
Inválida esperada	INEXISTENTE	POST userId inexistente en DB	ERROR SISTEMA	404 (real: 500 ■■■)
Válida	REGISTRADA	GET /attendances	REGISTRADA	200
Inválida no bloqueada	REGISTRADA	POST mismo userId mismo día	REGISTRADA	409 (real: 200 ■■■)

9.4 Casos de Prueba Derivados

A partir del diagrama de estados se derivarán los siguientes tipos de caso de prueba:

ID	Estado inicial	Acción	Entrada	Esperado	Real	Estado
ME-01	INEXISTENTE	POST /attendances	userId=2 (existente)	HTTP 200	HTTP 200	
ME-02	INEXISTENTE	POST /attendances	userId=null	HTTP 400	HTTP 400	
ME-03	INEXISTENTE	POST /attendances	userId=-1	HTTP 400	HTTP 400	
ME-04	INEXISTENTE	POST /attendances	userId=0	HTTP 400	HTTP 400	
ME-05	INEXISTENTE	POST /attendances	userId=99999	HTTP 404	HTTP 500	
ME-06	REGISTRADA	GET /attendances	—	HTTP 200	HTTP 200	
ME-07	REGISTRADA	POST mismo usuario	userId=2 (ya registrado)	HTTP 409	HTTP 200	

Resultado: 5 pasan, 2 fallan. Los fallos corresponden a defectos documentados en la Sección 6.4.

9.5 Implementación como Tests Automatizados

Cobertura	Resultado
State coverage (todos los estados alcanzados)	
Transition coverage (transiciones válidas ejecutadas)	
Invalid transitions tested	
Estados terminales verificados	

10. Conclusiones

10.1 Estado de los Entregables del Hito 4

El balance entregable por entregable al momento de cierre de esta versión del documento es el siguiente:

Entregable exigido por la consigna	Estado	Sección de referencia
Plan de Pruebas formal	Completo	Sección 2
Mínimo 15 casos de prueba diseñados con técnicas de caja negra	Excedido (36 casos)	Sección 3
Aplicación de Partición de Equivalencia y Valores Límite sobre ≥3 funciones	Excedido (5 campos en User + 4 en Activity + 1 en Attendance)	Secciones 3.2 a 3.4
Tabla de decisión (mínimo 1)	Completo (8 reglas, 4 casos)	Sección 3.5
Máquina de estados (mínimo 1)	En curso — aporte de Martín	Sección 9
Tests automatizados corriendo en verde	Completo (72 tests, 0 fallos)	Sección 4.2
Cobertura ≥60% en módulos principales	Excedido (100% en controller, service, model)	Sección 4.3 y 4.4
Mínimo 5 defectos documentados	Excedido (6 defectos)	Sección 6
Commits regulares y descriptivos	Completo (18 commits Hito 4)	Sección 7
Aplicación de TDD	Completo (2 ciclos Red-Green)	Sección 5
FMEA con mitigaciones	En curso — aporte de Fabricio	Sección 8

10.2 Métricas Finales del Avance

Métrica	Valor
Casos de prueba diseñados	36
Tests automatizados implementados	72
Tests automatizados pasando	72 / 72 (100%)
Defectos detectados y documentados	6
Defectos corregidos dentro del hito	4
Commits del Hito 4	18
Cobertura en módulos principales (controller, service, model)	100%
Cobertura global del backend	53.0%
Ciclos TDD completos (Red→Green) documentados	2

10.3 Lecciones Aprendidas

El avance del Hito 4 dejó tres aprendizajes principales que conviene preservar para los próximos hitos del proyecto:

El diseño dirigido por técnicas formales es eficiente: dedicar tiempo a enumerar clases de equivalencia y valores límite antes de escribir un solo test produjo 36 casos sólidos en una mañana de trabajo, todos justificables formalmente ante una defensa académica.

TDD acelera, no frena: aplicar el ciclo Red-Green sobre Activity y Attendance no agregó tiempo de trabajo respecto a hacerlo en orden tradicional, pero garantizó que el código nuevo naciera con tests, exposición formal de los defectos preexistentes, y el ciclo trazable en el historial de git.

La cobertura es un indicador, no un objetivo: lo importante no es el número global sino la cobertura sobre lo que realmente importa. Distinguir módulos principales de infraestructura de arranque permitió presentar la cobertura de manera honesta y defendible, en lugar de inflar el número con tests artificiales sobre clases de configuración.

10.4 Trabajo Pendiente para el Cierre Final

Antes de la entrega del 03/06/2026, restan integrar al documento:

La sección 8 (FMEA) con el aporte de Fabricio.

La sección 9 (Máquina de Estados) con el aporte de Martín, incluyendo la traducción del diseño a tests JUnit ejecutables.

Los defectos adicionales que surjan del testing exploratorio realizado por Ernesto, a integrar en la sección 6.

Una revisión cruzada por parte del equipo completo y un ensayo de la presentación oral.

Con estas piezas integradas, el documento quedará como el entregable consolidado del Hito 4, en condiciones de ser defendido ante la cátedra el 03/06/2026.

PARTE II — Actualización y Cierre (Hito 5)

Esta segunda parte continúa el mismo documento e integra el avance realizado durante el Hito 5: la corrección de defectos pendientes con TDD, la matriz de trazabilidad completa, las métricas finales, el análisis heurístico de usabilidad y la reflexión de cierre. Las secciones 1 a 10 reflejan el estado al Hito 4 y se conservan como base.

11. Resumen de Avance Hito 4 → Hito 5

Durante el Hito 5 se cerró el ciclo de calidad: se corrigieron defectos pendientes (incluido el crítico de control de roles) aplicando TDD, y se completó la documentación de trazabilidad, métricas y usabilidad exigida para la entrega final.

Aspecto	Hito 4	Hito 5
Tests automatizados	72	77
Defectos corregidos / cerrados	5	9
Defecto crítico RBAC (DEF-EXP-04)	Abierto	Corregido (TDD)
Cobertura módulos principales	100 %	100 %
Cobertura global	53,0 %	53,0 %
Pipeline CI	Verde	Verde

Documentación incorporada en el Hito 5:

- Matriz de Trazabilidad de Requerimientos (RTM) completa (sección 12).
- Métricas finales: LOC, complejidad ciclomática, mantenibilidad, cobertura y densidad de defectos (sección 13).
- Corrección de defectos con TDD, ciclo Red-Green visible en el historial (sección 14).
- Análisis heurístico de usabilidad según las 10 heurísticas de Nielsen (sección 15).
- Reflexión final del equipo (sección 16).

12. Matriz de Trazabilidad de Requerimientos (RTM)

Vincula cada requerimiento con su diseño, su implementación (backend y frontend) y los tests que lo verifican. Es la base de la trazabilidad exigida por el Plan SQA.

12.1 Requerimientos funcionales

ID	Requerimiento	Endpoint / Diseño	Backend	Frontend	Tests	Estado
REQ-F01	Registrar usuarios con nombre, email y DNI (validaciones)	POST /api/users	User (@NotBlank/@Size/@Email/@Pattern), UserController.createUser, UserService	UsersView.vue	UserControllerTest (CP-01..20, EX-01/02), UserServiceTest	OK
REQ-F02	Mostrar listado de usuarios registrados	GET /api/users	UserController.getAllUsers, UserService	UsersView.vue	UserControllerTest, UserServiceTest	OK
REQ-F03	Gestionar actividades (crear/editar/visualizar)	POST/PUT/GET /api/activities	Activity, ActivityController, ActivityService	ActivitiesView.vue	ActivityControllerTest (AC-01..11), ActivityServiceTest	OK

REQ-F04	Mostrar actividades disponibles	GET /api/activities	ActivityController.getAllActivities	ActivitiesView.vue	ActivityControllerTest, ActivityServiceTest	OK
REQ-F05	Registrar y mostrar historial de asistencia	POST /api/attendances	Attendance, AttendanceRequest, AttendanceController, AttendanceService	AttendanceView.vue	AttendanceControllerTest (AT-01..06), AttendanceServiceTest	OK

12.2 Requerimientos transversales (no funcionales)

ID	Requerimiento	Implementación	Test	Estado
REQ-NF01	Autenticación con JWT	AuthController, JwtUtil, JwtAuthenticationFilter, SecurityConfig	AuthControllerTest	OK
REQ-NF02	Autorización por rol (ADMIN/TRAINER/CLIENT)	User.role (@Pattern), MethodSecurityConfig, UserController (@PreAuthorize)	UserControllerSecurityTest (CLIENT 403 / ADMIN 200)	OK (DEF-EXP-04 corregido)
REQ-NF03	Manejo coherente de errores HTTP	GlobalExceptionHandler: 400/409/404/403	UserControllerTest (EX-01/02/03), UserControllerSecurityTest	OK

12.3 Cobertura de la trazabilidad

- 5 de 5 requerimientos funcionales implementados y con tests automatizados.
- Cada test referencia su caso de diseño (CP-, AC-, AT-, EX-).
- Trazabilidad inversa: cada defecto (DEF-XX) está vinculado al requerimiento y al test que lo expuso.

12.4 Mapa de pantallas (frontend) y requerimientos

Pantalla (frontend/src/views)	Requerimientos
LoginView.vue	REQ-NF01
DashboardView.vue	Resumen / navegación
UsersView.vue	REQ-F01, REQ-F02
ActivitiesView.vue	REQ-F03, REQ-F04
AttendanceView.vue	REQ-F05

13. Métricas Finales

Fecha de medición: 06/06/2026. Estado del repositorio: rama main, CI en verde.

13.1 Tamaño (LOC)

Componente	Archivos	LOC (sin líneas en blanco)
Backend — código fuente (backend/src/main)	24	704
Backend — tests (backend/src/test)	7	839
Frontend — código fuente (frontend/src)	11	881
Producción (backend + frontend)	35	1.585
Total con tests	42	2.424

Razón test/producción: $839 / 1.585 = 0,53$ (más de media línea de test por línea de producción).

13.2 Complejidad ciclomática (CC)

Medida con PMD 6.55.0 (regla CyclomaticComplexity, umbral 10).

Indicador	Valor
Violaciones de CC (métodos con CC > 10)	0
Métodos más complejos (capa service)	updateUser CC=3, updateActivity CC=2, registerAttendance CC=2
CC promedio estimada por método	~1,7

Todos los métodos cumplen $CC \leq 10$, el umbral del Plan SQA.

13.3 Maintainability Index (MI)

Fórmula SEI: $MI = 171 - 5,2 \cdot \ln(V) - 0,23 \cdot CC - 16,2 \cdot \ln(LOC)$. Con LOC/clase ≈ 29 , CC promedio 2 y Volumen de Halstead estimado ~ 340 , resulta $MI \approx 85$ (alta mantenibilidad en la escala SEI: ≥ 85 alta, 65–85 moderada, < 65 baja). El Volumen de Halstead se estima porque la toolchain actual (PMD, JaCoCo) no lo calcula; para un valor exacto se requeriría SonarQube o Radon.

13.4 Cobertura (JaCoCo)

Paquete	% líneas
controller/	100 %
service/	100 % (ramas 4/4)
model/	100 %
dto/	91,7 %
config/ (incluye GlobalExceptionHandler)	25,8 %
security/	2,2 %
TOTAL	53,0 %

Cobertura del 100 % en los módulos principales (controller, service, model), superando el umbral del Plan SQA (≥ 60 % en código nuevo). El 53 % global se explica por el paquete security/ y la configuración de arranque, fuera del alcance del testing funcional. Tests: 77 (0 fallos).

13.5 Densidad de defectos (defectos/KLOC)

Indicador	Valor
Defectos detectados	12 (6 documentados + 6 por testing exploratorio)
Defectos corregidos	9 (incluido el crítico de RBAC)
LOC de producción del backend	704 (0,704 KLOC)
Densidad sobre el backend	$12 / 0,704 \approx 17,0$ defectos/KLOC
Densidad sobre toda la producción	$12 / 1,585 \approx 7,6$ defectos/KLOC

Una densidad alta refleja un testing exhaustivo (se buscaron defectos de forma activa, incluido el exploratorio), no baja calidad: la mayoría son menores y están documentados.

13.6 Evolución Hito 3 → Hito 4 → Hito 5

Métrica	Hito 3	Hito 4	Hito 5
Tests automatizados	21	72	77
Cobertura global	28,3 %	53,0 %	53,0 %
Cobertura módulos principales	parcial	100 %	100 %
Defectos documentados	0	6	12
Defectos corregidos	0	5	9

Pipeline CI	no	sí	sí
-------------	----	----	----

13.7 Cumplimiento de umbrales del Plan SQA

Umbral	Objetivo	Resultado	Cumple
Cobertura módulos principales	$\geq 60 \%$	100 %	Sí
Complejidad ciclomática	$CC \leq 10$	0 violaciones	Sí
Tests en verde	100 %	77/77	Sí
Defectos críticos abiertos	0 al mergear	0	Sí

14. Corrección de Defectos en Hito 5 (TDD Red-Green)

En el Hito 5 se cerraron cuatro defectos detectados por testing exploratorio, cada uno con su ciclo Red-Green visible en el historial de commits (primero el test que falla, luego el fix).

ID	Descripción	Severidad	Estado
DEF-EXP-01	userId inexistente devolvía 500 en vez de 404	Menor	Corregido (Hito 5, TDD)
DEF-EXP-02	Se permitían asistencias duplicadas sin restricción	Mayor	Corregido (Hito 5, TDD)
DEF-EXP-04	Sin control de roles: CLIENT accedía a todas las rutas	Crítica	Corregido (Hito 5, TDD)
DEF-EXP-06	Día de actividad sin validación de formato en backend	Menor	Corregido (Hito 5, TDD)
DEF-EXP-03	No se puede editar usuario sin cambiar la contraseña	Mayor	Abierto
DEF-EXP-05	Mensaje incorrecto al eliminar usuario con asistencias	Menor	Abierto

Detalle de las correcciones:

- DEF-EXP-04 (crítico): RBAC con `@PreAuthorize("hasRole('ADMIN')")` — solo ADMIN gestiona usuarios; un CLIENT recibe 403 (MethodSecurityConfig + UserControllerSecurityTest).
- DEF-EXP-01: NotFoundException → 404 para usuario inexistente.
- DEF-EXP-02: rechazo de asistencia duplicada el mismo día → 409.

- DEF-EXP-06: validación @Pattern del día de la actividad → 400 si es inválido.

Estado consolidado: 12 defectos detectados, 9 corregidos/cerrados, 3 abiertos. Densidad $\approx$ 17 defectos/KLOC (backend), reflejo de un testing exhaustivo. Todos cargados como Issues en el repositorio con severidad, prioridad y trazabilidad.

15. Análisis Heurístico (10 heurísticas de Nielsen)

Evaluación de usabilidad de las pantallas (Login, Dashboard, Usuarios, Actividades y Asistencias) según las 10 heurísticas de Jakob Nielsen. Severidad: 1 cosmético, 2 menor, 3 mayor, 4 catastrófico.

15.1 Visibilidad del estado del sistema

- Fortaleza: los botones muestran "Guardando..." durante las operaciones; la navegación marca la vista activa.
- Problema (sev. 2): falta indicador de carga al traer los listados.
- Recomendación: agregar spinners/skeletons y un aviso de confirmación tras crear o editar.

15.2 Correspondencia con el mundo real

- Fortaleza: vocabulario del dominio gimnasio (Actividades, Asistencias, Costo mensual).
- Problema (sev. 1): algunos mensajes de validación muestran texto técnico (patrón del DNI).
- Recomendación: traducir los mensajes a lenguaje natural.

15.3 Control y libertad del usuario

- Fortaleza: botón Cancelar en los formularios y navegación libre por el menú.
- Problema (sev. 3): no hay confirmación al eliminar ni opción de deshacer; el borrado es irreversible.
- Recomendación: diálogo de confirmación antes de borrar y, de ser posible, baja lógica.

15.4 Consistencia y estándares

- Fortaleza: layout, botones y formularios con estilo consistente.
- Problema (sev. 1): conviene unificar color y posición de los botones de acción primaria.

- Recomendación: definir tokens de diseño reutilizados en todas las vistas.

15.5 Prevención de errores

- Fortaleza: validaciones en el backend (@Valid, @Pattern, @Size, @Positive).
- Problema (sev. 2): la validación de frontend es más débil que la de backend.
- Recomendación: replicar validaciones clave en el formulario y deshabilitar Guardar si es inválido.

15.6 Reconocer mejor que recordar

- Fortaleza: el día y el rol se eligen con un select (no texto libre).
- Problema (sev. 2): al registrar asistencia hay que conocer el userId; no hay buscador por nombre.
- Recomendación: selector con autocompletado de usuario por nombre.

15.7 Flexibilidad y eficiencia de uso

- Problema (sev. 2): no hay filtros, búsqueda ni paginación en los listados.
- Recomendación: agregar búsqueda, filtros y paginación en Usuarios y Actividades.

15.8 Diseño estético y minimalista

- Fortaleza: interfaz limpia, pocas pantallas y foco en la tarea.
- Problema (sev. 1): el Dashboard podría mostrar métricas útiles en lugar de quedar vacío.
- Recomendación: incorporar tarjetas de resumen en el Dashboard.

15.9 Reconocer, diagnosticar y recuperarse de errores

- Fortaleza: el GlobalExceptionHandler devuelve códigos HTTP coherentes (400, 409, 404, 403).
- Problema (sev. 3): el mensaje al eliminar un usuario con asistencias no corresponde al error real (DEF-EXP-05). El caso de userId inexistente ya se corrigió a 404 (DEF-EXP-01).
- Recomendación: corregir el mensaje de la eliminación con clave foránea y mostrarlo con claridad.

15.10 Ayuda y documentación

- Fortaleza: README completo con instalación y uso.
- Problema (sev. 2): no hay ayuda contextual ni tooltips para el usuario final.
- Recomendación: agregar tooltips en los campos no obvios y una breve guía por pantalla.

15.11 Resumen de hallazgos por severidad

Severidad	Cantidad	Ejemplos
Mayor (3)	2	sin confirmación al borrar; mensaje incorrecto al borrar usuario con asistencias
Menor (2)	5	sin indicador de carga; validación de frontend débil; sin filtros/paginación; sin buscador de usuario; sin ayuda contextual
Cosmético (1)	3	mensajes técnicos; consistencia de botones; Dashboard vacío

Prioridad de mejoras: (1) confirmación antes de eliminar; (2) corregir el mensaje de error al borrar con clave foránea (DEF-EXP-05); (3) buscador de usuario en el registro de asistencias. El control de roles (DEF-EXP-04), problema crítico de seguridad y usabilidad, ya se corrigió en el Hito 5.

16. Reflexión Final

16.1 Qué aprendimos

Migramos un sistema de gestión de gimnasio a una arquitectura moderna (Java + Spring Boot, Vue 3) y, sobre todo, incorporamos una cultura de calidad que al principio no teníamos:

- Testing como parte del desarrollo: pasamos de 21 a 77 tests y aplicamos TDD sobre funcionalidades nuevas.
- Técnicas de diseño de casos: partición de equivalencia, valores límite, tabla de decisión y máquina de estados.
- Métricas que importan: la cobertura es un indicador, no un objetivo; vale más el 100 % en módulos principales que un número global inflado.
- Integración continua: cada push corre los tests y avisa si algo se rompe.
- Trazabilidad: la RTM y los Issues dan visión clara de qué requerimiento cubre qué código y qué test.
- Trabajo en equipo con Git: convención de commits, ramas y revisión ordenaron el avance.

16.2 Qué haríamos distinto

- Empezar por los tests desde el día 1: varios defectos exploratorios no habrían llegado tan lejos.
- Implementar el control de roles (RBAC) antes; lo dejamos para el final y lo cerramos en el Hito 5.
- Reforzar la validación en el frontend, no solo en el backend.
- Cubrir también la capa de seguridad y el frontend con tests.
- Pulir la UX según el análisis heurístico: confirmación al borrar, búsqueda/paginación, mensajes claros.
- Cerrar el ciclo de los defectos abiertos antes de la entrega final.

16.3 Cierre

El proyecto nos dejó algo más valioso que la app en sí: un método de trabajo (planificar, testear, medir, documentar y automatizar) que podemos llevar a cualquier proyecto futuro.